TIPSY
AMAN KUMAR CHHARI

Building agents that **survive** real users.

ZERO TO BEST USE CASE · IN TYPESCRIPT

Nine phases for learning agentic AI properly — not fifteen half-finished demos. From the agent loop to guardrails, evals and a deployed system, built on the **OpenAI Agents SDK for TypeScript**.

Every phase ends with a checkpoint you can actually test yourself against. The theory is here, but the theory is not the point. **The shipped thing is the point.**

NODE 22+ · ZOD V4 · @OPENAI/AGENTS
COMPILED SEPTEMBER 2026

The route

No schedule, no deadlines. Go as slowly or as fast as the work allows. But the *order* matters more than it looks — each phase removes a specific failure mode that the phase before it introduces.

01	**The loop**	*an agent is not a function call*
02	**Tools**	*the description is the interface*
03	**Shape and state**	*typed output, hidden context*
04	**Many agents**	*handoffs versus managers*
05	**Guards**	*the loop needs a fence*
06	**The production layer**	*streaming, memory, MCP, traces*
07	**Proof and price**	*evals and economics*
08	**Shipping**	*a URL other people can open*
09	**Judgement**	*knowing when not to*

HOW EACH PHASE IS LAID OUT

Why it matters
What this phase buys you, and which specific mistake it stops you making.

What to study
Five core ideas, with what each actually means rather than what it is called.

The one line
The single piece of code that carries the phase. If you remember nothing else, remember this.

Read & follow
A short list, not a library. Three or four sources, all of them worth finishing.

Practice
One small exercise. Small enough to finish in a single sitting.

Checkpoint
A testable claim. If you cannot do the thing, do not move on — the next phase assumes it.

01 *an agent is not a function call*

The **loop**

WHY IT MATTERS

A chatbot takes input and returns output once. An agent receives an objective, decides what to do, acts, observes, and decides again — until it judges the job done. The model controls the flow at runtime; you do not. Every hard problem in the rest of this roadmap descends from that single sentence.

WHAT TO STUDY

The agent loop	Model call → tool call → result appended to history → model call again. Each pass is a *turn*, and every turn is a bill.
Instructions as policy	The system prompt is not a personality. It is the routing policy: scope, tool rules, and hard constraints, with constraints last.
maxTurns	The only thing standing between a confused agent and your entire budget. Set it to roughly twice the turns a correct run needs.
toolChoice & toolUseBehavior	Forcing tool use, and ending a run on a tool result instead of paying for one more model call to narrate it.
Reading traces	Tracing is on by default in Node. Learn to read a trace at the very start, before you have anything complicated to debug.

THE ONE LINE

```
import { Agent, run } from '@openai/agents';

const agent = new Agent({ name: 'Assistant', instructions: 'You are helpful.' });
const result = await run(agent, 'Write a haiku about recursion.', { maxTurns: 10 });
// result.finalOutput · result.history · result.newItems · result.state
```

READ & FOLLOW

- Agents SDK quickstart — openai.github.io/openai-agents-js
- “Building effective agents” — Anthropic engineering
- The SDK's `examples/agent-patterns` directory

PRACTICE

Build one agent with zero tools. Run it five times on the same input and read all five traces. Note where the outputs diverge — that variance is the thing the next eight phases exist to manage.

> **CHECKPOINT**
>
> You can explain, without notes, why an agent's cost is unpredictable and a chat completion's is not — and you have set `maxTurns` on every run you have written.

02 *the description is the interface*

Tools

WHY IT MATTERS

Tools are where an agent stops generating text and starts affecting the world. This is also where most agent bugs live — and almost none of them are in the implementation. The model chooses tools entirely from their names and descriptions. Your code is invisible to it.

WHAT TO STUDY

Function tools with Zod	One schema does three jobs: generates the JSON schema the model sees, validates arguments before your code runs, and types `execute`.
Writing descriptions	Say when to call it *and when not to*. The second half is what eliminates wrong-tool errors. Use `.describe()` on every non-obvious parameter.
Returning, not throwing	For expected failures, return a message the model can recover from. Reserve exceptions for genuine bugs.
Hosted tools	Web search, file search, code interpreter, image generation — run on OpenAI's side. `fileSearchTool` gives you RAG without building or hosting a vector store.
Timeouts and scope	Every network tool gets a timeout. One responsibility per tool — a `manage_order` with an action enum will be misrouted; three tools will not.

THE ONE LINE

```
const getOrder = tool({
  name: 'get_order',
  description: 'Fetch an order by ID. Returns status and date. Use before answering'
            + ' any order question. Does NOT include billing — use get_billing.',
  parameters: z.object({ orderId: z.string().describe('Order ID, not the email') }),
  timeout: 10_000,
  async execute({ orderId }) { /* return a string the model can reason over */ },
});
```

READ & FOLLOW

- Tools guide — Agents SDK docs
- Zod v4 docs — zod.dev
- OpenAI, “A practical guide to building agents”

PRACTICE

Write three tools for a fake shop. Then deliberately write one *bad* description, watch the agent misroute, and fix it. Doing that once teaches more than reading the whole guide.

> **CHECKPOINT**
> Given an agent that calls the wrong tool, your first instinct is to read the descriptions — not to blame the model or reach for a bigger one.

03 *typed output, hidden context*

Shape and **state**

WHY IT MATTERS

Two ideas that look like plumbing and are actually security and correctness. Structured output turns an agent into something the rest of your system can consume. Context turns identity into something the model cannot talk its way around.

WHAT TO STUDY

outputType	Pass a Zod schema and the SDK switches to structured outputs automatically. Stop prompting for JSON, stripping code fences, and writing parsers.
Schema design	Flat beats nested. Enums beat free strings for anything you branch on. Put `reasoning` *before* the answer — field order conditions the model.
Local context	`RunContext<T>` is dependency injection your tools read and the model never sees. DB handles, user IDs, feature flags.
The four ways in	Instructions, run input, tools, retrieval. Anything in the instructions is paid for on *every* turn; a tool is paid for only when called.
History	There is no implicit memory. `result.history` is how a conversation continues — and why long ones get expensive faster than you expect.

THE ONE LINE

```
// The model cannot see userId, cannot supply it, and cannot be talked into changing it.
const agent = new Agent<AppContext>({ name: 'Orders', tools: [fetchOrders] });

await run(agent, 'Show my recent orders', { context: { userId, db } });
// inside the tool:  runContext.context.userId   ← not a model-supplied argument
```

READ & FOLLOW

- Context management guide — Agents SDK docs
- Structured outputs — OpenAI platform docs
- Schema validation guide — Agents SDK docs

PRACTICE

Move a customer ID out of a tool's parameters and into context. Then actively try to talk the agent into fetching someone else's data. It should be impossible.

> **CHECKPOINT**
> You can state the rule in one sentence: if the model can name the resource, the model can name the wrong resource.

04 *handoffs versus managers*

Many **agents**

WHY IT MATTERS

One agent with twelve tools and a prompt full of “if the user asks about X” branches will be confused. Splitting it is not an optimisation — it is the fix. But there are two ways to split, and picking the wrong one costs you either your guardrails or your voice.

WHAT TO STUDY

Handoffs	A tool call that transfers the conversation. The specialist owns it from then on. Named `transfer_to_<agent>`; `handoffDescription` is your routing hint.
Managers (agents as tools)	`agent.asTool()`. The manager keeps control and summarises. One central place for guardrails, one consistent voice.
Choosing between them	If the specialist needs a *conversation*, hand off. If it needs to answer *one question* and return, make it a tool.
Input filters	By default the specialist inherits the entire history. `removeAllTools` strips tool-call noise — cheaper and it reasons better.
The deterministic option	Sometimes the flow is genuinely fixed. Calling `run()` three times in your own TypeScript is underrated and beats making the model rediscover a graph every time.

THE ONE LINE

```ts
// Agent.create — not new Agent — so TS infers the union of finalOutput shapes
const triage = Agent.create({
  name: 'Triage agent',
  instructions: `${RECOMMENDED_PROMPT_PREFIX}\nRoute billing vs refunds.`,
  handoffs: [billingAgent, handoff(refundAgent, { inputFilter: removeAllTools })],
});
```

READ & FOLLOW

- Agent orchestration guide — Agents SDK docs
- Anthropic, “Building effective agents” — the patterns vocabulary
- OpenAI cookbook — orchestration & handoffs

PRACTICE

Take your phase-02 shop agent and split it into triage plus three specialists. Measure the token count of one run before and after adding `removeAllTools` to a handoff.

CHECKPOINT

Given a workflow described in plain English, you can say which parts are handoffs, which are agent-tools, and which should not involve the model's judgement at all.

05 *the loop needs a fence*

Guards

WHY IT MATTERS

Your agent reads untrusted text and then takes actions. Text that looks like instructions can become instructions. There is no complete fix for prompt injection — there is a defence stack, and this phase is how you build it.

WHAT TO STUDY

The three families	Input guardrails run only on the *first* agent. Output guardrails only on the agent producing the *final* output. Tool guardrails run on *every* function-tool call. That asymmetry is the whole chapter.
runInParallel	`false` blocks before the model is called — no tokens, no tool execution. For a jailbreak filter, that is the entire point. `true` is lower latency and lets spend happen.
Guardrails as cheap agents	A small fast model with a boolean `outputType`, gating an expensive one. This is the standard shape.
Human-in-the-loop	`needsApproval` pauses the run and returns `interruptions`. Serialise `RunState`, store it, resume hours later in a different process.
The injection stack	Fewest tools in scope · identity from context · approval on anything irreversible · guardrail the boundary · keep untrusted content in *input*, never instructions.

THE ONE LINE

```
const jailbreakGuard: InputGuardrail = {
  name: 'jailbreak',
  runInParallel: false,          // block BEFORE the expensive model runs
  execute: async ({ input, context }) => {
    const r = await run(cheapClassifier, input, { context });
    return { outputInfo: r.finalOutput, tripwireTriggered: r.finalOutput.blocked };
  },
};
```

READ & FOLLOW

- Guardrails & human-in-the-loop guides — Agents SDK docs
- OWASP Top 10 for LLM Applications
- NIST AI Risk Management Framework — overview

PRACTICE

Put `needsApproval: true` on a refund tool. Approve from the terminal. Then serialise the state to a file, kill the process, restart, and resume the same run.

> **CHECKPOINT**
>
> You can name, for any agent you have built, the exact set of irreversible actions it can take — and every one of them is behind an approval.

06 *streaming, memory, MCP, traces*

The production **layer**

WHY IT MATTERS

Everything so far works on your laptop. This phase is the difference between that and something a stranger will tolerate using: visible progress, memory that does not grow forever, tools you did not write, and a way to see what happened.

WHAT TO STUDY

Streaming	Users tolerate slow. They do not tolerate blank. Stream text *and* the event stream — "Searching the web…", "Handing off to refunds" — then always await `stream.completed`.
Sessions and compaction	Four options: `result.history`, a `Session`, `conversationId`, `previousResponseId`. Compaction rewrites long history — run it off the critical path or it hangs the end of a stream.
Three memory layers	Working memory (last N turns) · summary (compaction) · *facts in your own database*. The third is the one people skip and rebuild badly later.
MCP	Stdio, Streamable HTTP, or hosted. Filter the tool list — a server exposing forty tools will wreck your routing. Treat every MCP result as untrusted text.
Tracing	On by default in Node. Group multi-step workflows with `withTrace`. On serverless, `forceFlush()` before teardown or your traces vanish in prod and work locally.

THE ONE LINE

```
const stream = await run(agent, input, { stream: true, session });

for await (const event of stream) {
  if (event.type === 'agent_updated_stream_event') ui.showHandoff(event);
  if (event.type === 'run_item_stream_event')      ui.showToolCall(event);
}
await stream.completed;   // session persistence finishes AFTER the last token
```

READ & FOLLOW

- Streaming, Sessions, MCP, Tracing guides — Agents SDK docs
- modelcontextprotocol.io — the spec, it is short
- ByteByteGo — for the systems vocabulary around all of this

PRACTICE

Attach a filesystem MCP server holding three policy documents, filter it down to two tools, and stream the answer to a terminal with tool-call status lines.

> **CHECKPOINT**
> Your agent shows the user what it is doing while it does it, and you can open a trace and explain any run from last week.

07 *evals and economics*

Proof and **price**

WHY IT MATTERS

This is the phase almost every tutorial omits and the one that separates a demo from a product. Prompt edits are code changes with no type checker. Without evals you will accumulate twenty prompts you are afraid to touch.

WHAT TO STUDY

Golden datasets	Twenty cases beats zero, and zero is what you have. Sources in priority order: every bug you have ever hit · real user inputs · adversarial inputs · happy paths last.
Property assertions	Assert which tools were called, which were *not*, which agent finished, how many turns. Never assert exact output strings — that test will be flaky and you will delete it.
LLM-as-judge	For tone and faithfulness. Two rules: reasoning field before the verdict, and a *stronger* model than the one being judged.
Where money goes	Every turn re-sends the whole history. Tool schemas ride along on every call. Guardrail agents are extra calls. Reasoning tokens are billed and invisible.
The five failure causes	Bad tool description · agent has too many jobs · missing context · schema too complex · model not capable enough. Check that last one *last*. It is rarely true.

THE ONE LINE

```
const toolsCalled = result.newItems
  .filter(i => i.type === 'tool_call_item')
  .map(i => i.rawItem.name);

expect(toolsCalled).toContain('get_order');        // property, not string
expect(toolsCalled).not.toContain('process_refund');
```

READ & FOLLOW

- *AI Engineering* — Chip Huyen (the evals chapters)
- Braintrust or LangSmith docs — dataset versioning
- OpenAI evals — platform docs

PRACTICE

Write twenty cases from bugs you already hit in phases 01–06. Wire them into CI. Gate on “no regression against last pass rate”, not on 100% — runs are non-deterministic.

> **CHECKPOINT**
>
> You can change a prompt and know, before you commit it, whether you made things better or worse. And you can state your cost per run to the nearest paisa.

08 *a URL other people can open*

Shipping

WHY IT MATTERS

A deployed narrow agent teaches you things a local broad one cannot. Ship with three tools rather than stopping at phase 05 with twelve. The hard problems only appear under traffic you did not write yourself.

WHAT TO STUDY

Server-only execution	Route handlers or server actions. `runtime = 'nodejs'`, not edge. The key never reaches the browser, and auth is checked before the agent runs.
AI SDK UI bridge	`@openai/agents-extensions/ai-sdk-ui` turns an Agents stream into a standard AI SDK streaming `Response`, so the React hooks work with no adapter of your own.
Durable execution	Anything over about a minute belongs on a queue. Trigger.dev or Inngest give you retries, scheduling and observability for runs that must survive a restart.
Serverless HITL	The run pauses, you serialise state to your DB, the function returns. The approval arrives minutes later as a separate request and a fresh instance resumes.
What to watch	Turns per run · tool error rate · guardrail trip rate · p95 by agent. Alert on turns-per-run: it moves before your users complain.

THE ONE LINE

```ts
// app/api/chat/route.ts
export const runtime = 'nodejs';

export async function POST(req: Request) {
  const session = await auth();
  if (!session) return new Response('Unauthorized', { status: 401 });
  const stream = await run(agent, message, { stream: true, maxTurns: 10,
                                     context: { userId: session.user.id } });
  return createAiSdkUiMessageStreamResponse(stream);
}
```

READ & FOLLOW

- Next.js 15 route handlers & streaming docs
- Trigger.dev — OpenAI Agents SDK playground
- Zerodha, Razorpay, Swiggy engineering blogs

PRACTICE

Deploy the shop agent. Give the URL to three people who did not build it. Add every input that breaks it to your eval set. Repeat until nothing new breaks.

> **CHECKPOINT**
>
> Someone who is not you has used your agent, and you found out about the resulting bug from a dashboard rather than from them.

09 *knowing when not to*

Judgement

WHY IT MATTERS

The most valuable thing an agent engineer can say is “this does not need an agent.” A dropdown, a cron job, or forty lines of `if` will beat an LLM on cost, latency and reliability far more often than the discourse suggests. Being able to argue *against* a feature is what makes your argument *for* one credible.

WHAT TO STUDY

The simplest pattern that works	Prompt → RAG → single tool call → agent loop → multi-agent. Climb only when the rung below demonstrably fails. Most production “agents” should be rung two or three.
Where AI earns its cost	High volume, messy input, clear success criteria, and tolerable error. Remove any one of those four and reconsider.
Where it does not	Low volume, structured input, zero error tolerance, or a task a form already solves. Also: anything where a wrong answer is expensive and unverifiable.
Cost per task vs value per task	Compute both before building. If the agent costs ₹4 per run and saves ₹3 of human time, you have built an expensive demo.
Explaining it	Answer first, support after. If a shopkeeper cannot repeat your explanation of what the agent does back to you, you do not have a product yet.

THE ONE QUESTION

```
// Before writing any agent code, answer this in one paragraph:
//
//   What is the cheapest thing that would work?
//   Why is it not good enough?
//   What does a wrong answer cost, and who notices?
//
// If you cannot answer the third one, you are not ready to build it.
```

READ & FOLLOW

- *Good Strategy / Bad Strategy* — Richard Rumelt
- *The Design of Everyday Things* — Don Norman
- Stratechery — for how technical strategy gets argued

PRACTICE

Take one real process — a clinic's appointment booking, a kirana's order intake — and write a one-page assessment: should it use an agent, which pattern, rough running cost, main risk.

> **CHECKPOINT**
>
> You have talked at least one person — possibly yourself — out of building an agent, and you were right to.

Small **habits**

This sticks through repetition, not intensity. None of these are scheduled — each one is tied to a trigger, so it happens whenever the trigger does. The most valuable of them is reading traces, and it is the one people skip.

TRIGGERS

Before writing code	Answer the three questions from phase 09: what is the cheapest thing that would work, why is it not enough, what does a wrong answer cost.
After every run	Open the trace. Not the logs — the trace. Ask why the model chose what it chose, not just what it returned.
After every bug	Add it to the eval set before you fix it. This is non-negotiable and it is the cheapest insurance in the whole roadmap.
After every surprise	Write 300 words explaining what surprised you to someone non-technical. If you cannot, you did not understand it.
Before every phase	Re-read the previous phase's checkpoint and prove it to yourself. The next phase assumes it.
Whenever you use an AI product	Run the teardown below on it. No code, just thinking — and it compounds faster than anything else here.

THE AGENT TEARDOWN TEMPLATE

Use this on any AI product you open — Cursor, Perplexity, a bank's support bot, a WhatsApp ordering flow.

01 · The job	What task is it doing, for whom, and how often? Is that volume high enough to justify a model?
02 · The pattern	Prompt, RAG, single tool call, loop, or multi-agent? Could it be one rung lower?
03 · The tools	What can it actually *do* to the world? That set is its blast radius if someone injects it.
04 · The fence	Where does it ask permission? What does it refuse? Try to make it misbehave and watch what catches you.
05 · The seams	Where does it feel slow, repeat itself, or lose the thread? Those are the memory and streaming decisions showing.
06 · Your call	If they asked you, what one change would you make — and what would it cost them per run?

WORTH FOLLOWING

The Agents SDK `examples/` directory for working code · Anthropic and OpenAI engineering blogs for patterns · Latent Space and Chip Huyen for the field's actual state · Zerodha and Razorpay engineering for how Indian systems get built · Finshots for the business side of whatever you are about to automate.

The use case **atlas**

Problems, sorted by the shape of the problem rather than the industry. Find the row your situation matches, build the pattern in the second column, and do not climb higher until it fails you.

THE PROBLEM SHAPE	REACH FOR	WHY THAT, AND NOT MORE
Rewrite, classify or extract from one piece of text	**A prompt with `outputType`**	No loop, no tools, one call, typed result. If this works, you are done — most “agent” demos are secretly this.
Answer questions over a fixed document set	**RAG via `fileSearchTool`**	Retrieval is a lookup, not a decision. An agent loop adds cost and latency and changes nothing.
Answer needs one live fact from your system	**One function tool, `stop_on_first_tool`**	The tool result *is* the answer. Skipping the narration call halves your cost at scale.
Multi-step research with unknown depth	**Agent loop + web search + `maxTurns`**	The step count genuinely cannot be known in advance. This is the first honest use of a loop.
Support across several distinct domains	**Triage → handoffs**	Keeps each prompt short and each specialist independently reasonable. One long branching prompt gets confused.
Several independent subtasks, one voice	**Manager + `agent.asTool()`**	Guardrails live in one place, the model can fan out in a single turn, and the user hears one consistent voice.
Output quality matters more than latency	**Evaluator–optimizer loop**	A critic agent with explicit criteria, capped iterations. Expensive; markedly better on writing and code.
Reads a codebase, runs tests, edits files	**Sandbox agent** (beta)	Genuine need for a workspace and shell. Container it, give it no credentials, allowlist the commands.
Spoken interaction, phone or browser	**`RealtimeAgent` + WebRTC**	A persistent low-latency session with interruption handling. A request/response loop cannot do this.
Action is irreversible or moves money	**Any of the above + `needsApproval`**	Not a pattern so much as a requirement. An injection that triggers an approval prompt is a caught injection.
The steps are actually always the same	**Three `run()` calls in your own code**	Do not make the model rediscover a fixed graph on every request. You are paying for nondeterminism you did not want.
Low volume, structured input, zero error tolerance	**A form. Seriously.**	Nothing here beats a dropdown and a validation rule. Being able to say this is phase 09.

The **bookshelf**

Three sources covering the three sides of this work: the system underneath, the discipline of shipping models, and the patterns that give you a shared vocabulary with everyone else building agents right now.

Designing Data-Intensive Applications

PHASE 06

Martin Kleppmann · the system underneath

What it is	How real systems store, move and reconcile data — replication, consistency, queues, and what breaks under load.
How it helps	An agent is a distributed system with a weird scheduler. Sessions, durable execution, retries and serialised run state all become obvious once you have this vocabulary.

AI Engineering

PHASE 07

Chip Huyen · the discipline

What it is	Building applications on foundation models: evaluation, dataset construction, cost, latency, and the difference between a demo and a system.
How it helps	This is the book that makes you take evals seriously before you have been burned. Read the evaluation chapters twice.

Building Effective Agents

PHASES 01 & 04

Anthropic engineering · the patterns

What it is	A short essay naming the patterns the whole field now uses: prompt chaining, routing, parallelisation, orchestrator-workers, evaluator-optimizer.
How it helps	Gives you the words for what you are building, and makes a strong case for the simplest pattern that works. Short, and you will keep returning to it.

Cheat **sheet**

SETUP

```
npm install @openai/agents zod
# Node 22+ · Zod v4
export OPENAI_API_KEY=sk-...
export DEBUG=openai-agents*
```

CORE API

```
new Agent({ name, instructions, model,
  tools, handoffs, outputType,
  inputGuardrails, outputGuardrails,
  modelSettings, toolUseBehavior,
  mcpServers })

Agent.create({...})  // with handoffs
agent.clone({...})   // shares arrays!
agent.asTool({ toolName, toolDescription })

tool({ name, description, parameters,
  execute, needsApproval, isEnabled,
  timeout, inputGuardrails })

run(agent, input, { context, maxTurns,
  stream, session })
```

RESULT

```
result.finalOutput   result.history
result.newItems      result.state
result.interruptions result.completed
```

MODEL SETTINGS

```
toolChoice: 'auto' | 'required'
          | 'none' | 'my_tool'
toolUseBehavior: 'run_llm_again'
  | 'stop_on_first_tool'
  | { stopAtToolNames: [...] }
resetToolChoice: true  // stops loops
retry: { policies: [
  retryPolicies.networkError(),
  retryPolicies.httpStatus([429,500]) ] }
```

HOSTED TOOLS

```
webSearchTool()
fileSearchTool({ vectorStoreIds })
codeInterpreterTool()
imageGenerationTool()
hostedMcpTool({ serverLabel, serverUrl })
computerTool() shellTool()
applyPatchTool()  // sandbox these
```

ERRORS YOU WILL MEET

```
MaxTurnsExceededError
ToolCallError · ToolTimeoutError
InputGuardrailTripwireTriggered
OutputGuardrailTripwireTriggered
ModelBehaviorError // bad args → flatten schema
ModelRefusalError · UserError
```

PRE-LAUNCH CHECKLIST

☐ **Approvals**	Every tool that writes, sends or deletes
☐ **Identity**	No tool takes a user ID from the model
☐ **maxTurns**	Set on every entry point
☐ **Timeouts**	On every network tool
☐ **Guardrails**	Input with `runInParallel: false`
☐ **Rate limits**	Per user, upstream of the agent
☐ **Logging**	Usage per run, sensitive logging off
☐ **Traces**	`forceFlush()` on serverless

Ship the narrow thing. **Then widen it.**

Three failure modes account for almost everyone who sets out to learn this and stalls.

Building broad before building deep. Fifteen half-finished demos teach less than one deployed agent with three tools. The hard problems only appear under real traffic.

Skipping evals until "later". Later never comes, and by then you have twenty prompts you are afraid to change because you cannot tell whether an edit helped. Twenty cases written before you need them are worth more than two hundred written after.

Treating tool descriptions as documentation. They are the agent's entire decision-making surface. When something misbehaves, read the descriptions first.

Everything else is detail you can look up.